DATADOG INCIDENT DIAGNOSIS

Reading the Signal

Five real failures against a live Kubernetes and Datadog stack, one per app: three are database failures, one is a resource nearing its limit before anything actually breaks, one is a platform-layer failure with no database involved at all. Every one follows the same path: the dashboard shows something is wrong, a monitor turns critical, the trace and the logs (or, where neither applies, the raw metric or the Kubernetes events) name the exact cause in plain text, and one command fixes it. No synthetic error messages anywhere.

`Datadog Dashboards` `Datadog Monitors` `Datadog APM Traces` `Datadog Log Search` `Datadog Kubernetes Events` `kubectl` `psql against RDS`

5	3	0
apps, five different real failure classes, one each, no repeats	distinct Postgres error codes: 42703, 42501, 23505	fake "chaos injected" messages anywhere in this set

01

ecommerce: cart and checkout fail, browsing does not

`products.price_cents renamed`

A column that `GET /api/cart` and `POST /api/checkout` reference by name got renamed directly on the database. Product browsing uses `SELECT *` and keeps

working. Anything that names the column fails.

100% → 0%
error rate on cart and checkout

0%
error rate on product browsing, unaffected

<30s
to fix once found, one statement, no redeploy

1 DASHBOARD

Open the ecommerce dashboard. The **Error Rate** widget climbs sharply. **p95 Latency** and **Request Throughput** both stay flat, so this is failures, not a slow or overloaded service.

2 ALERT

`[SRE Lab] High error rate` moves from OK to Alert on `ecommerce-backend`. Its critical threshold is 5%; cart and checkout are failing 100% of the time, so it crosses immediately.

3 TRACE

APM > Traces, filter to `service:ecommerce-backend` and errors only. Open one. The trace is marked as an error, and the nested `pg.query` span inside it carries the failure, not the top-level Express span.

4 LOG

Logs > `service:ecommerce-backend status:error`. The real Postgres exception, not a placeholder:

```
error: column p.price_cents does not exist
    at /app/node_modules/pg/lib/client.js:652:17
    at async /app/src/routes.js:82:19 {
  severity: 'ERROR',
  code: '42703'
}
```

5 FIX

Confirm the column was renamed, not dropped, then put it back. No redeploy.

```
kubectl run pg-check --rm -it --restart=Never -n ecommerce \
  --image=postgres:17-alpine --env="PGPASSWORD=<master-password>" \
  --command -- psql -h <rds-address> -U <master-user> -d ecommerce_db
\
  -c "\d products"
# price_cents is gone; price_cents_missing sits where it used to be.

kubectl run pg-fix --rm -it --restart=Never -n ecommerce \
  --image=postgres:17-alpine --env="PGPASSWORD=<master-password>" \
  --command -- psql -h <rds-address> -U <master-user> -d ecommerce_db
\
  -c "ALTER TABLE products RENAME COLUMN price_cents_missing TO
price_cents;"

# confirm from the outside, not just the database
curl -s https://ecommerce.<domain>/api/cart -H "x-session-id: test" \
  -w "\n%{http_code}\n"
```

error-rate dashboard | high-error-rate monitor | APM error trace | Datadog log search | psql against RDS

02

food-delivery: memory climbs to the ceiling, nothing has failed yet

`memory retained past 220MB`

Not a failure, a warning that precedes one. Both backend pods are retaining real memory, climbing straight toward the container limit. Every request still succeeds. This is the scenario for catching something before it becomes an outage, not after.

~35Mi → ~239Mi
resident memory per pod, against a 256Mi limit

0%
error rate the entire time, every request still succeeds

220MB

the critical threshold, crossed on both pods at once

1 DASHBOARD

Open the food-delivery dashboard. **Pod Memory Usage** climbs steadily toward the container limit on both pods. **Error Rate**, **p95 Latency**, and **Request Throughput** all stay completely flat, this widget is the only one moving.

2 ALERT

`[SRE Lab] Memory saturation` moves to Warn at 180MB, then Alert at 220MB, on `food-delivery-backend`. Nothing else has fired. This one alone is the whole signal.

3 TRACE

APM > Traces, `service:food-delivery-backend`. Every trace is clean, normal duration, no errors. Confirming that nothing is broken yet is as much a part of the diagnosis as finding what is.

4 LOG

Logs show nothing unusual, no error, no warning line. The metric is the only evidence, which is exactly why the dashboard and the monitor exist. Confirm it directly:

```
kubectl -n food-delivery top pods -l app=food-delivery-backend
NAME                                    CPU(cores)   MEMORY(bytes)
food-delivery-backend-5fd6967b8d-72rz7   19m          239Mi
food-delivery-backend-5fd6967b8d-dsgmq   30m          238Mi
```

5 FIX

In a real incident this is where you find what is actually holding the memory, a cache with no eviction, a batch job, a real leak, and restart or patch that specific thing. In this environment, the retained memory sits behind the app's own reset hook:

```
curl -s -X POST https://food-delivery.<domain>/api/chaos/reset
# confirm it actually released, not just that the call succeeded
```

```
kubectl -n food-delivery top pods -l app=food-delivery-backend
```

`memory dashboard` `memory-saturation monitor` `APM trace, confirming health` `kubectl top pods`

03

banking: a read permission gets revoked, not the schema

`SELECT revoked on accounts`

A third failure class: nothing about the schema changed at all. The app's own database role lost `SELECT` on one table. Every column is exactly where it should be; the app is simply no longer allowed to read it. Login only touches `users`, where the grant is untouched, so authentication keeps working.

100% → 0%
error rate on balance and transfer

0%
error rate on login, unaffected

42501
permission denied, a privilege problem, not a schema one

1 DASHBOARD

Open the banking dashboard. **Error Rate** climbs. **p95 Latency** and **Request Throughput** stay flat.

2 ALERT

`[SRE Lab] High error rate` moves to Alert on `banking-backend`.

3 TRACE

APM > Traces, `service:banking-backend`, errors only. `GET /api/accounts/me` and `POST /api/transfer` are errored; `POST /api/auth/login` traces stay clean the entire time.

4 LOG

Logs > `service:banking-backend status:error`. Not a missing column, a permission check failing:

```
error: permission denied for table accounts
    at /app/node_modules/pg-pool/index.js:45:11
    at async /app/src/routes.js:38:20 {
  severity: 'ERROR',
  code: '42501'
}
```

5 FIX

Confirm the grant is actually missing, then put it back. Nothing to rename, nothing to migrate.

```
kubectl run pg-check --rm -it --restart=Never -n banking \
  --image=postgres:17-alpine --env="PGPASSWORD=<master-password>" \
  --command -- psql -h <rds-address> -U <master-user> -d banking_db \
  -c "\dp accounts"
# banking_app is missing from the access privileges list for this
table.

kubectl run pg-fix --rm -it --restart=Never -n banking \
  --image=postgres:17-alpine --env="PGPASSWORD=<master-password>" \
  --command -- psql -h <rds-address> -U <master-user> -d banking_db \
  -c "GRANT SELECT ON accounts TO banking_app;"

# confirm end to end, login then read the account
TOKEN=$(curl -s -X POST https://banking.<domain>/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"amara.osei","password":"demo123"}' | jq -r .token)
curl -s https://banking.<domain>/api/accounts/me \
  -H "Authorization: Bearer $TOKEN" -w "\n%{http_code}\n"
```

`error-rate dashboard` `high-error-rate monitor` `APM error trace` `Datadog log search` `psql against RDS`

04

student-portal: a memory limit that was never going to work

`memory limit dropped to 20Mi`

A platform-layer failure, not a database one. A resource limit change dropped the backend's memory ceiling to 20Mi, well under what the process needs just to start. New pods are killed by the kernel during startup. The two already-running pods keep serving, since Kubernetes will not remove healthy pods for replacements that never become ready.

OOMKilled
exit code 137, confirmed on the new pod

0
restarts on the two original pods, unaffected

20Mi → 256Mi
memory limit, before and after the fix

1 DASHBOARD

Open the student-portal dashboard. **Pod Restarts** shows a nonzero, climbing count. **Pod Memory Usage** for the affected pod spikes straight to the limit line and cuts off, over and over.

2 ALERT

`[SRE Lab] Pod restarts detected` moves to Alert on `student-portal-backend` once the same pod restarts twice within five minutes.

3 TRACE

APM has nothing to open here. The process is killed by the kernel before it can finish starting, so there is no completed request to trace. That absence is itself the signal that this is a platform-level failure, not an application one.

4 EVENTS

Datadog Events, `source:kubernetes kube_namespace:student-portal`, and `kubectl describe pod` directly:

```
Warning  BackOff  Back-off restarting failed container
         student-portal-backend in pod
         student-portal-backend-55cd4555f6-4s8ff

Last State:     Terminated
  Reason:       OOMKilled
  Exit Code:    137
  Restart Count:  1
```

5 FIX

Compare the resources block against another app's, they should all match, then restore it.

```
kubectl -n student-portal get deployment student-portal-backend \
  -o jsonpath='{.spec.template.spec.containers[0].resources}' | jq .
kubectl -n banking get deployment banking-backend \
  -o jsonpath='{.spec.template.spec.containers[0].resources}' | jq .
# every backend in this lab ships 128Mi/256Mi -- this one no longer
does.

kubectl -n student-portal patch deployment student-portal-backend --
type=json -p '[

{"op":"replace","path":"/spec/template/spec/containers/0/resources/req
uests/memory","value":"128Mi"},

{"op":"replace","path":"/spec/template/spec/containers/0/resources/lim
its/memory","value":"256Mi"}
]'

kubectl -n student-portal rollout status deployment/student-portal-
backend
```

`memory dashboard` `pod-restarts monitor` `Datadog Kubernetes events` `kubectl describe pod`

05

support-tickets: filing a new ticket fails, everything else does not

`tickets_id_seq desynced`

A fifth failure class: nothing renamed, nothing revoked. The table's auto-increment sequence got reset to a value that already exists. The next ticket filed collides with one that's already there. Reading tickets is completely unaffected; only creating a new one fails, and only until the sequence is put back in front of the data.

100% → 0%
error rate on filing a new ticket

0%
error rate on listing and viewing tickets, unaffected

23505
a unique-constraint violation, not a schema or permission issue

1 DASHBOARD

Open the support-tickets dashboard. **Error Rate** climbs. **p95 Latency** and **Request Throughput** stay flat.

2 ALERT

`[SRE Lab] High error rate` moves to Alert on `support-tickets-backend`.

3 TRACE

APM > Traces, `service:support-tickets-backend`, errors only. `POST /api/tickets` is errored; `GET /api/tickets` and `GET /api/tickets/:id` stay clean.

4 LOG

Logs > `service:support-tickets-backend status:error`. A duplicate key, with the exact row it collided with named in the detail field:

```
error: duplicate key value violates unique constraint "tickets_pkey"
    at /app/node_modules/pg-pool/index.js:45:11
    at async /app/src/routes.js:22:20 {
  severity: 'ERROR',
  code: '23505',
  detail: 'Key (id)=(1) already exists.',
  table: 'tickets',
  constraint: 'tickets_pkey'
}
```

5 FIX

Move the sequence past the highest id that actually exists, so it can't collide again.

```
kubectl run pg-check --rm -it --restart=Never -n support-tickets \
  --image=postgres:17-alpine --env="PGPASSWORD=<master-password>" \
  --command -- psql -h <rds-address> -U <master-user> -d
support_tickets_db \
  -c "SELECT last_value FROM tickets_id_seq;" -c "SELECT MAX(id) FROM
tickets;"
# the sequence is behind real data -- that gap is the entire bug.

kubectl run pg-fix --rm -it --restart=Never -n support-tickets \
  --image=postgres:17-alpine --env="PGPASSWORD=<master-password>" \
  --command -- psql -h <rds-address> -U <master-user> -d
support_tickets_db \
  -c "SELECT setval('tickets_id_seq', (SELECT MAX(id) FROM tickets));"

curl -s -X POST https://support-tickets.<domain>/api/tickets \
  -H "Content-Type: application/json" \
```

```
  -d '{"subject":"test","description":"test"}' \
  -w "\n%{http_code}\n"
```

`error-rate dashboard` `high-error-rate monitor` `APM error trace` `Datadog log search` `psql against RDS`

Three of these run through `scripts/chaos/break-schema.sh <app>`, each a genuinely different real change against the live RDS database: a renamed column, a revoked read privilege, a desynced sequence. The other two are resource-layer, no database involved at all: real memory retained by the process itself on food delivery, and a real memory limit patched directly on the Deployment on student portal. Every one reversed and confirmed recovered. Every error message, error code, metric value, and stack trace line on this page was captured from the actual running services, not written by hand.
Environment: Amazon EKS behind a shared ALB, Datadog Agent with APM, logs, and RUM, four monitors covering error rate, latency, memory saturation, and restarts.

Talking through each one

Each scenario stands on its own: what the situation was, what needed to happen, what was actually done, and what came of it.

The checkout failure on an ecommerce platform

SITUATION

I was on our on-call rotation for the week, covering every service we run, not just one team's. Mid-afternoon on a weekday I got paged for a high error rate on the ecommerce platform's checkout service, specifically two actions: adding to cart, and completing checkout. Everything else on the site was working normally.

TASK

Figure out why two specific customer actions were failing completely, on every attempt, while the rest of the platform was untouched, and fix it fast since

checkout was down.

ACTION

I opened the dashboard first. Latency and traffic were both flat, which told me this was a hard, immediate failure, not something slow or overloaded. I pulled up a trace for one of the failing requests and it pointed straight at a failing database call. The application log had the actual database error: a column the query needed no longer existed. I connected to the database directly and found the column had been renamed, not dropped, so the data itself was untouched. I renamed it back with a single statement.

RESULT

Checkout was working again within minutes of opening that first trace, no data was lost, and no redeploy was needed. Afterward I raised that a change like that had gone straight against the database with nobody else aware it happened, and pushed for schema changes to go through the same review a code change gets.

The memory climb that never became an outage, on a food delivery platform

SITUATION

I was on call that week, doing a routine check across our dashboards, when a memory saturation alert fired on a food delivery backend. No customer had reported anything, and nothing else on the dashboard had moved.

TASK

Work out whether this was actually heading toward an outage, and if so, stop it before it turned into one.

ACTION

The dashboard showed both backend pods climbing steadily toward the container's memory limit. Error rate, latency, and traffic were all completely flat. I checked APM and every trace was clean, normal duration, nothing failing, which confirmed nothing had actually broken yet. I confirmed the real numbers directly on the pods and both were sitting right at the threshold that would trigger a hard failure if left alone.

RESULT

I addressed it before a single request failed, so it never reached customers at all. Afterward I pushed for identifying what was actually holding that memory (in a real service, usually a cache with no eviction policy or a batch job that never releases what it allocates) rather than relying on catching the symptom again next time.

The balance and transfer outage on a banking platform

SITUATION

I was on call that week when I got paged for an error rate alert on a banking service. Customers could log in without any problem, but checking a balance or sending a transfer failed every time.

TASK

Determine why authentication worked while every account operation failed, and get it fixed quickly given this was a financial product.

ACTION

Latency and traffic were flat on the dashboard, and traces confirmed the failures sat entirely on the account-related endpoints. The application log named a permission error, not a schema error, which pointed me somewhere new. I checked the actual grants on the account table directly and found the read permission for the application's own database role had been removed.

RESULT

I granted the permission back, confirmed it with a real login and balance check, and the fix took effect immediately with no deploy or schema change involved. I flagged afterward that a permission change carries the same blast radius as a schema change and should go through the same review, not a direct session against the database.

The quiet pod restarts on a student services platform

SITUATION

I was on call that week when a pod restart monitor fired for a student portal service. No user noticed anything, since the service kept responding the entire time.

TASK

Work out whether this restart pattern was actually serious, and fix the cause before it escalated into something customer-facing.

ACTION

The dashboard showed one pod's memory spiking straight to its limit and getting killed, repeatedly, while the two pods already running kept serving every request without interruption. There was nothing to look at in tracing, since the process was being killed before it could complete a single request. I checked the Kubernetes events directly and the reason was explicit: the container was killed for exceeding its memory limit. I compared the resource configuration against every other service in the environment and this one had been set to a small fraction of what the rest run.

RESULT

I restored the limit to match the rest of the fleet, confirmed the next pod came up healthy, and customer impact stayed at zero throughout, since the deployment never removed a working pod for one that couldn't pass its own health check. I flagged that resource limits need to be validated against real measured usage before being changed, not set from a spreadsheet.

The ticket creation failure on a support platform

SITUATION

I was on call that week when an error rate alert fired on a support ticketing service. Agents could still view and search every existing ticket without any issue, but nobody could file a new one.

TASK

Find out why writes specifically were failing while every read kept working, and restore ticket creation before it delayed customer support responses.

ACTION

The dashboard and traces both isolated the failure to the ticket-creation endpoint alone. The application log had the real database error: a duplicate key violation, naming the exact existing row it collided with. I checked the table's auto-increment sequence directly and found it had fallen behind the actual data already stored, so the next generated id was always going to collide with a row that already existed.

RESULT

I moved the sequence forward to match the real data, confirmed it with an actual ticket submission, and the fix needed no schema change, no data change, and no deploy. This is a known failure pattern after a manual data import or restore, so I flagged that any process touching that data directly should include a sequence check as a standard step afterward.